Dual-View Address Mapping for NPU–PIM Memory on Ramulator 2

Yash Gupta
Roll No. 264101064
Dept. of Computer Science and Engineering
IIT Guwahati
yash.g@iitg.ac.in

Nikunj Ladumor
Roll No. 264101032
Dept. of Computer Science and Engineering
IIT Guwahati
nikunj.ladumor@iitg.ac.in

Vamshi Kumar Kanduri
Roll No. 264101061
Dept. of Computer Science and Engineering
IIT Guwahati
vamshi.kanduri@iitg.ac.in

Priyanshu Ganjeer
Roll No. 264101035
Dept. of Computer Science and Engineering
IIT Guwahati
priyanshu.ganje@iitg.ac.in

***Abstract*—An NPU and a processing-in-memory (PIM) unit that share one DRAM want different physical layouts of the same weight matrix: the NPU streams the matrix and needs consecutive accesses spread over independent DRAM resources, while all-bank PIM needs the parallel PIM units to sit in different banks over the same input range. PFM [1] chooses one jointly optimized layout per 2 MB page with a mixed-integer linear program (MILP). We reproduce this optimizer for the weight shapes of LLaMA3-8B. A PuLP/CBC MILP assigns the 15 line-index bits of a 2 MB page to column, bank-group, bank and row bits of a DDR4 device under capacity and structural constraints, maximizing $\mathrm{NPU}(f)+\alpha\,\mathrm{PIM}(f)$. We add a region-aware address mapper to Ramulator 2 that gives every 2 MB region its own layout. We compare NPU-friendly, PIM-friendly and jointly optimized layouts: NPU sequential-read utilization is simulated in Ramulator, and PIM matrix-vector (GEMV) utilization is computed with an analytical all-bank model driven by Ramulator's DDR4-2400 timings. Over all seven weight matrices of a decoder layer, the optimized layout with α=0.25 keeps 93.8% NPU utilization (NPU-friendly: 95.2%) while raising PIM utilization from 18.6% to 48.1%; with α=2 it reaches 96.1% PIM utilization (PIM-friendly: 71.3%). For `down_proj`, both fixed layouts (designed for d_{in}=4096) give 4% PIM utilization and the optimizer recovers 48–96%. We also show that on single-channel DDR4 no layout can be best for both devices at once, and explain why.**

***Index Terms*—processing-in-memory, NPU, DRAM address mapping, LLM inference, integer linear programming, Ramulator**

I. Introduction

LLM inference mixes compute-bound phases (prefill, batched GEMM) with memory-bound phases (decode-time GEMV). Recent systems pair an NPU for the former with near-bank PIM for the latter [2]–[4]. When both share one memory, the weight matrices exist only once, so they have one physical layout. The two devices want different layouts [1], [5]:

- The **NPU** reads a matrix as one sequential stream. It is fast when consecutive bursts go to independent resources (channels, or bank groups in DDR4) so that the data bus never waits.
- **All-bank PIM** [6], [7] sends one command to all banks; each bank's PIM unit processes a different output column over the same input slice. This needs the units' columns in *different banks* and each column's data in as few rows as possible.

PFM [1] solves this with a *dual-view* memory: one physical layout per 2 MB page, chosen by a MILP that scores how well a layout serves each device.

Task. We implement the PFM address-mapping optimizer for one tensor shape in Python with PuLP [8], add a mapping choice per memory region to Ramulator 2 [9], and compare three layouts for the LLaMA3-8B [10] weight matrices: NPU-friendly, PIM-friendly and jointly optimized. We report bandwidth utilization for sequential NPU reads and for PIM GEMV. GPU kernels, hardware synthesis and larger models are out of scope.

Contributions. (1) A verified MILP that maps 15 address bits to DDR4 levels and matches the best of 20,000–100,000 random layouts in every case. (2) `PFMRegionMapper`, a new Ramulator 2 address mapper that holds a table of layouts and selects one per 2 MB region, checked to give cycle-identical results to an independent Python bit permutation, and a utilization metric checked against Ramulator's own throughput counter. (3) Measurements on all seven LLaMA3-8B weight matrices for DDR4 x8 and x16, an α sweep, and a bound that explains why one layout cannot serve both devices fully on single-channel DDR4.

II. Background

A. DRAM organization and timing

A DDR4 address selects a row, a column, a bank and a bank group [11]. A read to the open row of a bank is a row hit; a read to another row needs precharge and activate ($t_{RP}+t_{RCD}=16+16$ cycles for DDR4-2400R). Two reads to the *same* bank group must be $t_{CCD_L}=6$ cycles apart; reads to *different* bank groups only $t_{CCD_S}=4$ cycles. A 64 B burst

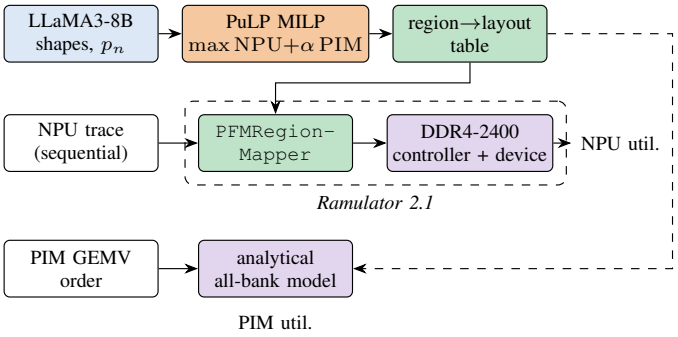


Fig. 1. Flow. The MILP picks a layout per tensor class; inside Ramulator, `PFMRegionMapper` maps every 2 MB region with its own layout. The PIM model uses the same layout table.

(BL8) occupies the bus for 4 cycles, so a stream that stays in one bank group can reach at most $4/6 = 66.7\%$ of the bus bandwidth. On DDR4 the bank-group bits therefore play the role that channel bits play in the HBM system of the paper.

B. Address mapping

A mapping assigns each physical-address bit to one DRAM level. Inside a 2 MB page of 64 B lines there are $2^{21}/2^6 = 2^{15}$ lines, i.e. 15 free bits. For DDR4_8Gb_x8 (4 bank groups $\times$ 4 banks) they split into 7 column, 2 bank-group, 2 bank and 4 row bits; for x16 (2 bank groups) into 7, 1, 2 and 5.

C. Weight layout and the PIM unit window

We compute $y = xW$ with FP16 weights stored column by column, so one column of W is $d_{\text{in}} \cdot 2\,\text{B} = 2^{p_n}$ lines (p_n=7 for d_{in}=4096; p_n=9 for d_{in}=14336 padded to 16384). Line $\ell = c \cdot 2^{p_n} + t$ holds offset t of column c. With U PIM units (one per bank: U=16 for x8, U=8 for x16), unit u processes columns $c = Ug + u$. The bits that select the unit are therefore line-index bits $p_n \ldots p_n + \log_2 U - 1$, which we call the *unit window*. All-bank PIM runs at full speed only if every unit-window bit is a bank-group or bank bit.

III. Method

Fig. 1 shows the flow. The experiments are about 700 lines of Python; the mapper is about 160 lines of C++.

A. Per-region mapping and simulator setup

A layout is a list of (level, bit count) pairs from the least significant line-index bit upward, e.g. `bg1 col6 bank1 col1 bank1 bg1 row4`. It is a pure bit permutation (no XOR, no dropped bits).

`PFMRegionMapper`. We added a new address-mapper implementation to Ramulator 2 (one C++ file registered with the controller's `addr_mapper` interface, plus one line in `CMakeLists.txt`; the Python wrapper is generated by Ramulator). Its parameters are `region_bits` (21 for 2 MB), a list of `layouts`, a `region_layout` table with one layout id per region, and a `default_layout` for regions outside the table (-1 = stock RoBaRaCoCh). For each request it takes the line index, splits it into a region number and 15 local bits, sends every local bit to the DRAM level and bit position given by that region's layout, and places the region number above the layout's row bits. At start-up it checks that every layout gives the column, bank-group and bank levels exactly their device bit counts. Weights of the same d_{in} share one layout.

Independent check. We also implemented the same permutation in Python: it rewrites each address so that Ramulator's stock RoBaRaCoCh mapper decodes the intended fields, and asserts that all 2^{15} lines map to distinct addresses. For all 20 layouts and both access patterns, the two paths give identical cycle counts (Section IV-F).

Ramulator 2 runs one DDR4-2400R channel; we use version 2.1 (commit `c6c7a5d` [12]), the successor of Ramulator 2.0 [9] in the same code base. It is configured as DDR4_8Gb_x8 or x16 with one rank, FR-FCFS scheduling, open-row policy and all-bank refresh. For the Python path we located the RoBaRaCoCh field positions (x8: column [12:6], bank group [14:13], bank [16:15], row [32:17]) and verified them by simulation (Fig. 3). Every run also asserts that the trace has no duplicate addresses and that all reads reach DRAM.

Metric. NPU utilization $= 4\,N_{\text{reads}}/\text{cycles}$, with cycles in DRAM clocks. On 8192 plain sequential reads this gives 65.9%, identical to Ramulator's reported throughput divided by the 19.2 GB/s peak (12,659.6 MB/s / 19,200 MB/s).

B. MILP formulation

Binary variable $x_{l,j} = 1$ if line-index bit $j \in \{0..14\}$ selects level $l \in \{\text{Co}, \text{BG}, \text{Ba}, \text{Ro}\}$. Following PFM (whose capacity constraint is $\sum_j x_{l,j} \le B_l$; because the device bit counts add up to exactly 15, every level must be filled and the bound holds with equality):

$$\textstyle\sum_l x_{l,j} = 1 \quad \forall j \qquad \text{(structural)} \qquad (1)$$

$$\textstyle\sum_j x_{l,j} = B_l \quad \forall l \qquad \text{(capacity)} \qquad (2)$$

NPU score. A bank-group bit at position 0 makes consecutive reads alternate bank groups (t_{CCD_S}); a row bit below a column bit makes a sequential sweep reopen rows:

$$\text{NPU}(f) = \tfrac{2}{3} + \tfrac{1}{3} x_{\text{BG},0} - \lambda \frac{\sum_{i<j} z_{ij}}{B_{\text{Ro}} B_{\text{Co}}}, \quad z_{ij} \ge x_{\text{Ro},i} + x_{\text{Co},j} - 1, \qquad (3)$$

with $\lambda = 0.25$; $2/3$ and 1 are the bus rates under t_{CCD_L} and t_{CCD_S}.

PIM score. Let k be the number of bank-group or bank bits and r the number of row bits in the unit window of width $w = \log_2 U$. With $k < w$, 2^{w-k} units share each bank and serialize; row bits in the window add row switches:

$$\text{PIM}(f) = 2^{-(w-k)} \cdot 0.5^{\,r}. \qquad (4)$$

This term is made linear with a one-hot binary $y_{k,r}$ over all feasible (k, r) pairs. The objective is $\max\ \text{NPU}(f) + \alpha\,\text{PIM}(f)$ plus a 10^{-4} tie-break that pushes row bits up.

Verification. For every solve we check that (a) the MILP objective equals an exact Python evaluation of the returned layout (catches linearization errors) and (b) it is at least as

TABLE I

LAYOUTS OF THE 15 LINE-INDEX BITS OF A 2 MB PAGE (DDR4 X8), MSB ON THE LEFT. CO: COLUMN, BG: BANK GROUP, BA: BANK, RO: ROW; THE NUMBER IS THE BIT COUNT. $\alpha \geq 1$ GIVES THE SAME LAYOUT AS $\alpha=1$.

Case	Layout	Bits 14 → 0
x8, d_{in}=4096	NPU-friendly	`Ro4 Ba2 Co7 BG2`
x8, d_{in}=4096	PIM-friendly	`Ro4 Ba2 BG2 Co7`
x8, d_{in}=4096	PFM α=0.25	`Ro4 BG1 Ba1 Co1 Ba1 Co6 BG1`
x8, d_{in}=4096	PFM α=1	`Ro4 BG1 Ba2 BG1 Co7`
x8, d_{in}=14336	NPU-friendly	`Ro4 Ba2 Co7 BG2`
x8, d_{in}=14336	PIM-friendly	`Ro4 Ba2 BG2 Co7`
x8, d_{in}=14336	PFM α=0.25	`Ro2 BG1 Ba2 Co1 Ro2 Co6 BG1`
x8, d_{in}=14336	PFM α=1	`Ro2 Ba1 BG2 Ba1 Ro2 Co7`

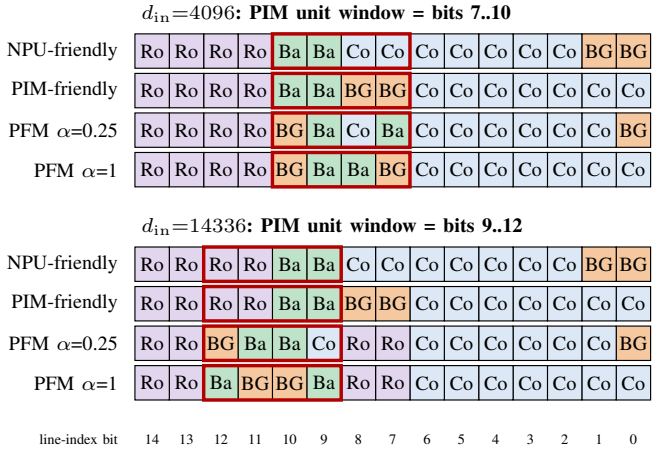


Fig. 2. Bit layouts on DDR4 x8 (bit 14 left). The red box marks the PIM unit window. The PFM layouts keep a bank-group bit at bit 0 for the NPU (α=0.25) or fill the window with bank-level bits (α=1); for d_{in}=14336 they move row bits out of the window.

good as the best of 20,000 random layouts (100,000 in the first model). Both held in all cases. Re-running the solver reproduces every stored layout exactly.

C. PIM GEMV model

Stock Ramulator 2 has no PIM commands, so PIM utilization comes from an analytical model that uses Ramulator's DDR4-2400R values. Each step issues one all-bank column command: units $u = 0..U-1$ read offset t of their current column. A unit costs $t_{CCD_L} = 6$ cycles, plus $t_{RCD} = 16$ if its bank has no open row or $t_{RP} + t_{RCD} = 32$ if the row changes. Units in the same bank add up, and the step lasts as long as the slowest bank. Utilization is the ideal time (6 cycles per step) over the modeled time. As a cross-check, we also replay the same access order as ordinary reads through Ramulator (the "read-stream proxy").

D. Layouts compared

NPU-friendly: BG lowest, then Co, Ba, Ro. **PIM-friendly**: Co lowest, then BG, Ba, Ro. **PFM**: the MILP solution for $\alpha \in \{0.25, 1, 2\}$ and each p_n. Table I and Fig. 2 list them.

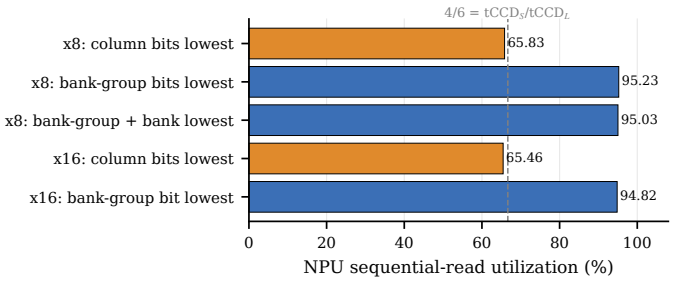


Fig. 3. NPU sequential-read utilization in Ramulator for simple layouts. With column bits lowest every read stays in one bank group and pays t_{CCD_L}; one bank-group bit at the bottom is enough to reach 95%.

IV. RESULTS

A. Bank-group interleaving decides NPU bandwidth

Fig. 3 shows the basic effect. With column bits lowest, utilization is 65.8% (x8) and 65.5% (x16), just under the $4/6$ bound; with a bank-group bit lowest it is 95.2% and 94.8%. Row hits stay above 99% in both cases, so the loss comes from t_{CCD_L}, not from row misses. Adding bank bits below the bank-group bits gives nothing (95.0%) and three times more row conflicts. One bank-group bit is enough because two alternating groups already space same-group reads by 8 cycles $> t_{CCD_L}$.

B. Baselines

The first two bars of each group in Fig. 4 show the two fixed layouts. NPU-friendly reaches 95% NPU utilization but only 24% PIM utilization on x8 with d_{in}=4096: two of its four unit-window bits are column bits, so four units share each bank. PIM-friendly reaches 96% PIM but only 66% NPU. For d_{in}=14336 (p_n=9) the unit window moves up to bits 9–12, which in both fixed layouts contain two row bits. Four units then share a bank *and* use different rows, so every access is a row switch and PIM utilization falls to 3.95%. Note that both fixed layouts are designed for d_{in}=4096; a PIM-friendly layout designed by hand for d_{in}=14336 would move the row bits out of bits 9–12, which is what the optimizer does for $\alpha \geq 1$.

C. Jointly optimized layouts

The other two bars and Table II add the MILP layouts.

- d_{in}=4096 **(six matrices).** PFM α=0.25 keeps the bank-group bit at position 0 and puts the other three bank-level bits in the unit window: 95.2% NPU (same as NPU-friendly) with 48.1% PIM instead of 24.0%. For $\alpha \geq 1$ the solver picks a layout equivalent to PIM-friendly (only the order of bank-level bits inside the window differs), with the same utilization.
- d_{in}=14336 **(`down_proj`).** PFM moves two row bits to positions 7–8 (the top of the within-column offset), out of the unit window. With α=0.25 the window holds three bank-level bits and one column bit (one bank-group bit stays at bit 0 for the NPU), giving 90.2% NPU

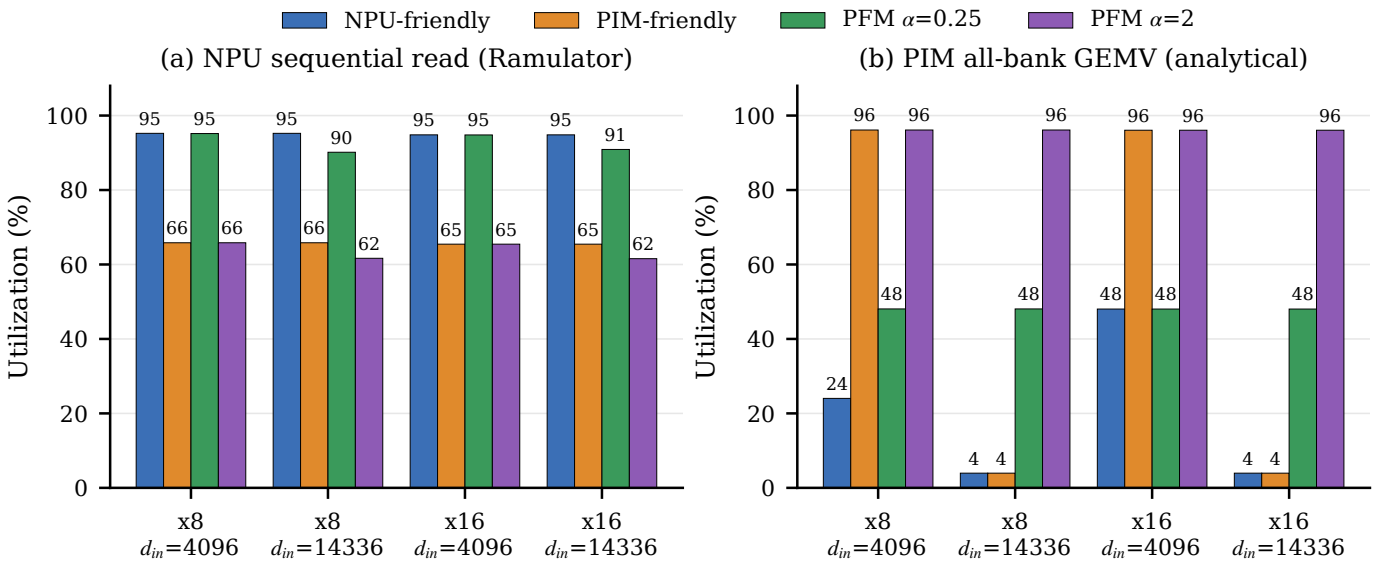


Fig. 4. Bandwidth utilization of all layouts; the first two bars of each group are the baselines. PFM with α=0.25 keeps NPU utilization near the NPU-friendly layout and doubles (x8, d_{in}=4096) or multiplies by 12 (d_{in}=14336) the PIM utilization; α=2 gives the best PIM utilization in every case.

TABLE II
BANDWIDTH UTILIZATION (%) OF EVERY LAYOUT. NPU: RAMULATOR. PIM: ANALYTICAL ALL-BANK MODEL; THE READ-STREAM PROXY IS SHOWN FOR REFERENCE.

Case	Layout	NPU	PIM	PIM proxy
x8, d_{in}=4096	NPU-friendly	95.23	24.03	91.25
	PIM-friendly	65.83	96.12	95.03
	PFM α=0.25	95.17	48.06	94.87
	PFM α=2	65.83	96.12	95.03
x8, d_{in}=14336	NPU-friendly	95.23	3.95	51.66
	PIM-friendly	65.83	3.95	27.85
	PFM α=0.25	90.15	48.06	94.91
	PFM α=2	61.66	96.12	94.98
x16, d_{in}=4096	NPU-friendly	94.82	48.03	94.83
	PIM-friendly	65.46	96.06	94.86
	PFM α=0.25	94.79	48.03	94.83
	PFM α=2	65.46	96.06	94.86
x16, d_{in}=14336	NPU-friendly	94.82	3.95	53.03
	PIM-friendly	65.46	3.95	36.66
	PFM α=0.25	90.90	48.03	94.83
	PFM α=2	61.56	96.06	94.85

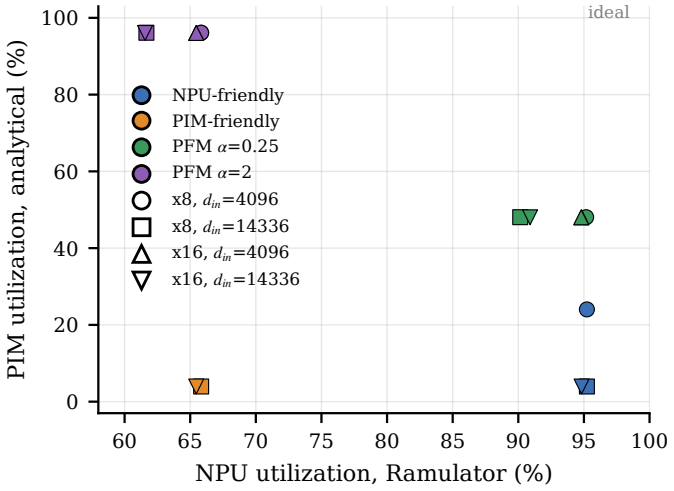


Fig. 5. NPU vs. PIM utilization for all layouts. PFM layouts match or improve on the fixed layouts of the same case, but none reaches the top-right corner on single-channel DDR4.

and 48.1% PIM; with $\alpha \geq 1$ it holds all four bank-level bits, giving 96.1% PIM at 61.7% NPU. Both fixed layouts give 3.95% PIM. The read-stream proxy agrees in direction (51.7% $\rightarrow$ 94.9%). Much of this gain comes from adapting the layout to the column length, which a hand-tuned PIM layout for this shape could also do; the optimizer's added value here is that it finds the shape-specific layout automatically and, for α=0.25, one that also keeps 90% NPU utilization.

- **x16** shows the same pattern (Table II).

D. Whole decoder layer

Six of the seven matrices (d_{in}=4096; 152 pages) share the first case and `down_proj` (56 pages of weight data) the second. Weighting each matrix by its size (Fig. 6): on x8, PFM α=0.25 gives up 1.4 NPU points against NPU-friendly (93.8% vs. 95.2%) and gains 29.4 PIM points (48.1% vs. 18.6%). PFM α=2 gains 24.8 PIM points over PIM-friendly (96.1% vs. 71.3%) for 1.1 NPU points.

E. Effect of α and the single-layout bound

For d_{in}=4096 the solver switches from the NPU-oriented to the PIM-oriented layout between α=0.25 and α=1. In the model the switch is at $\alpha = 2/3$, where $1 + 0.5\alpha = 2/3 + \alpha$.

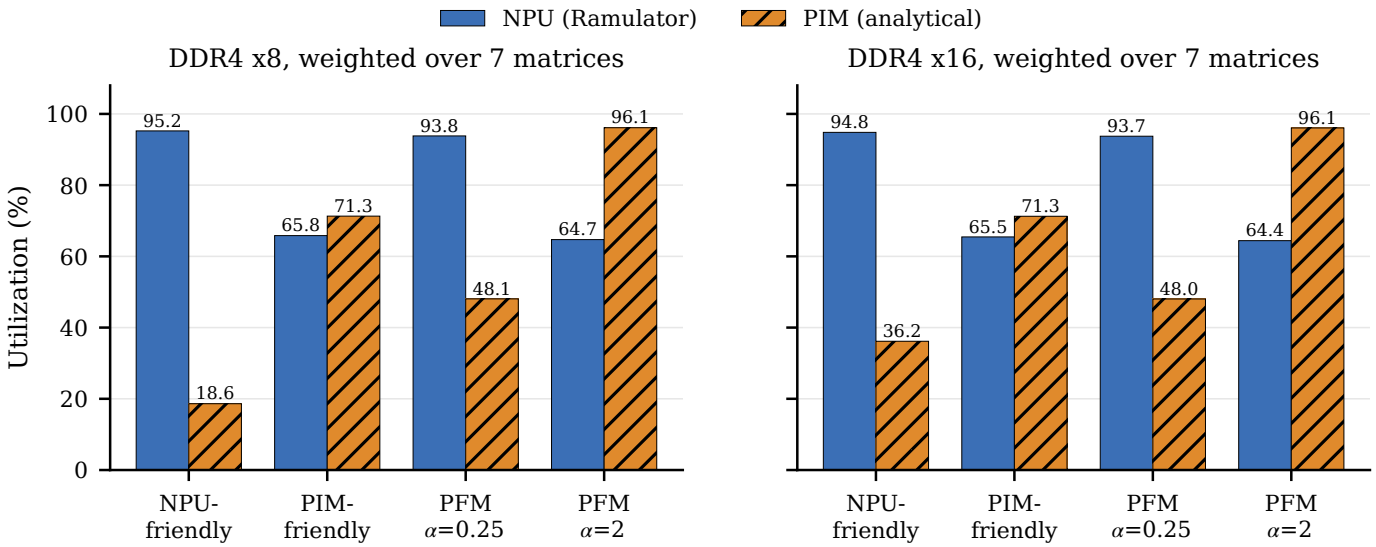


Fig. 6. Utilization over all seven LLaMA3-8B weight matrices of one decoder layer, weighted by their size in 2 MB pages of weight data.

TABLE III
ONE NPU STREAM OVER EIGHT 2 MB REGIONS (SIX d_{in}=4096 PAGES, TWO `down_proj` PAGES) WITH `PFMRegionMapper` INSIDE RAMULATOR (DDR4 X8). THE REGION TABLE GIVES THE LAYOUT ID OF EACH REGION.

Plan	Layouts	Region table	NPU %
NPU-friendly everywhere	1	00000000	95.05
PIM-friendly everywhere	1	00000000	65.81
PFM α=0.25 per region	2	00000011	93.70
PFM α=2 per region	2	00000011	64.70

A low α suits prefill-heavy serving, a high α decode-heavy serving.

No layout reaches high utilization on both devices on DDR4 x8, and this follows from the bit budget. The NPU needs one bank-group bit at position 0. The unit window needs all four bank-level bits (w=4), but x8 has only four, so with one at position 0 at most three can lie in the window ($k \leq 3$), which caps PIM at $2^{-1} = 50\%$ of peak. The paper's HBM2e target [1] has separate channel and pseudo-channel bits for NPU interleaving, so it does not face this conflict; in our reading, this is why PFM can report high utilization on both devices while our DDR4 study shows a trade-off.

F. Per-region mapping inside Ramulator

For 20 of 20 layouts (both DRAM parts, both d_{in} classes), replaying the plain NPU and PIM traces through `PFMRegionMapper` gives exactly the same cycle count as the Python permutation with RoBaRaCoCh, and the same utilization as Table II. Table III then uses several regions in one simulation: a single NPU stream reads eight 2 MB regions, six holding d_{in}=4096 weights and two holding `down_proj`. With the PFM layouts the region table selects layout 0 for the first six regions and layout 1 for the last two. The α=0.25 plan reaches 93.7% NPU utilization, close to the page-weighted value of the single-region runs (93.9%), so switching layouts between regions costs almost nothing.

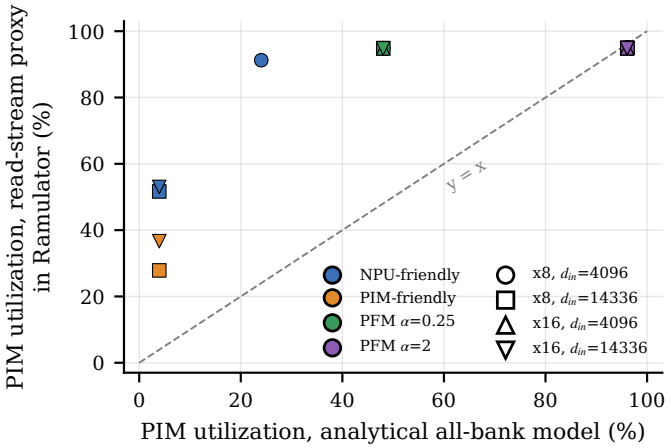


Fig. 7. Read-stream proxy in Ramulator vs. analytical all-bank model. FR-FCFS reorders ordinary reads and hides unit-to-bank conflicts, so the proxy is close to 95% for most layouts.

G. Model fidelity

The NPU score tracks Ramulator closely: 1.00 vs. 95.2%, 0.67 vs. 65.8%, and 0.98 vs. 90.2% for the `down_proj` layout with row bits at positions 7–8; scaled by the 95.2% that a score of 1.00 gives, 0.98 predicts about 93.3%, so the score is about 3 points optimistic there. For PIM, the read-stream proxy and the analytical model disagree (Fig. 7): the scheduler reorders independent reads and finds other banks, which a lock-step all-bank PIM command cannot do. We therefore report the analytical model for PIM and keep the proxy as a lower-fidelity check.

V. Limitations

1) PIM utilization is analytical, not simulated. The optimizer's PIM score and the model share the assumption that unit-window bits must be bank-level bits, so their agreement shows consistency rather than independent validation.
2) The memory is single-channel DDR4, not multi-channel HBM2e-PIM; bank-group bits stand in for channel bits, and the paper's numbers (its Table III) are not reproduced.
3) The fixed baselines are the usual NPU-friendly and PIM-friendly orders for d_{in}=4096; for `down_proj` they are weak, so the gain there partly reflects shape adaptation rather than joint optimization.
4) The whole-layer numbers weight `down_proj` by its 56 pages of weight data; with each column padded to 512 lines, its layout would occupy 64 pages. Using 64 changes the weighted utilizations by at most 2.5 points and changes no ranking.
5) Layouts are static per tensor class; there is no runtime scheduler, no address-translation hardware and no validation against real hardware.

VI. Conclusion

We reproduced the PFM address-mapping optimizer as a verified PuLP MILP and evaluated it on the LLaMA3-8B weight shapes with Ramulator 2. A new Ramulator address mapper, `PFMRegionMapper`, gives every 2 MB region its own layout and reproduces an independent Python implementation cycle for cycle. The jointly optimized layouts improve PIM utilization at nearly the same NPU utilization (18.6%→48.1% at 93.8% NPU over a decoder layer), and they turn `down_proj` from unusable for PIM (4%) into 48–96%. On single-channel DDR4 the bit budget forces a trade-off that α selects. Next steps are multiple channels and PIM commands inside Ramulator for a cycle-level PIM timing model. Source code: https://github.com/groupiitg/PFM.

References

[1] S. Zhao, L. Liu, T. Han, M. Wang, Y. Han, and Y. Wang, "Rethinking unified memory for NPU-PIM systems: Dual-view memory for dynamic inference of LLM," arXiv:2608.06989 [cs.AR], 2026.

[2] G. Heo *et al.*, "NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inferencing," in *Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[3] M. Seo *et al.*, "IANUS: Integrated accelerator based on NPU-PIM unified memory system," in *Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[4] J. Park *et al.*, "AttAcc! unleashing the power of PIM for batched transformer-based generative model inference," in *Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[5] S. H. Seo, J. Kim, D. Lee, S. Yoo, S. Moon, Y. Park, and J. W. Lee, "FACIL: Flexible DRAM address mapping for SoC-PIM cooperative on-device LLM inference," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2025.

[6] S. Lee *et al.*, "Hardware architecture and software stack for PIM based on commercial DRAM technology," in *Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2021.

[7] M. He *et al.*, "Newton: A DRAM-maker's accelerator-in-memory (AiM) architecture for machine learning," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2020.

[8] S. Mitchell, M. O'Sullivan, and I. Dunning, "PuLP: A linear programming toolkit for Python," The University of Auckland, Tech. Rep., 2011.

[9] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible DRAM simulator," *IEEE Computer Architecture Letters*, vol. 23, no. 1, pp. 112–116, 2024.

[10] Llama Team, AI @ Meta, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[11] JEDEC, "DDR4 SDRAM standard," JEDEC Standard JESD79-4.

[12] H. Luo, F. N. Bostancı, A. Olgun, M. Makeenkova, Z. Malik, I. Akdeniz, and O. Mutlu, "Ramulator 2.1: A composable memory system simulator for modern DRAM systems," The 3rd Tutorial on Ramulator and DRAM Bender, colocated with ICS, 2026.